



Claude Code는 안다고 가정 — forge는 처음인 개발자를 위해

forge 사용법

상황별 워크플로우로 익히는 18개 fg 스킬

오늘의 목표

강의가 끝나면 여러분은 할 수 있다

- 1 상황을 보고 어떤 fg 명령을 쓸지 판단한다
- 2 ask → run → next 한 바퀴를 직접 돌린다
- 3 재진입·실패·무인 주행을 스스로 처리한다
- 4 언제 fg-quick을 쓰고 언제 정식 루프로 갈지 고른다

표기와 전제

명령 표기

본문은 README 공식 표기 `/fg-*` 를 씁니다. 환경에 따라 `/forge:fg-ask` 처럼 보일 수 있습니다 — 같은 명령입니다.

대상

Claude Code 기초(프롬프트·도구·서브에이전트)는 안다고 가정합니다. 그 부분은 다루지 않습니다.

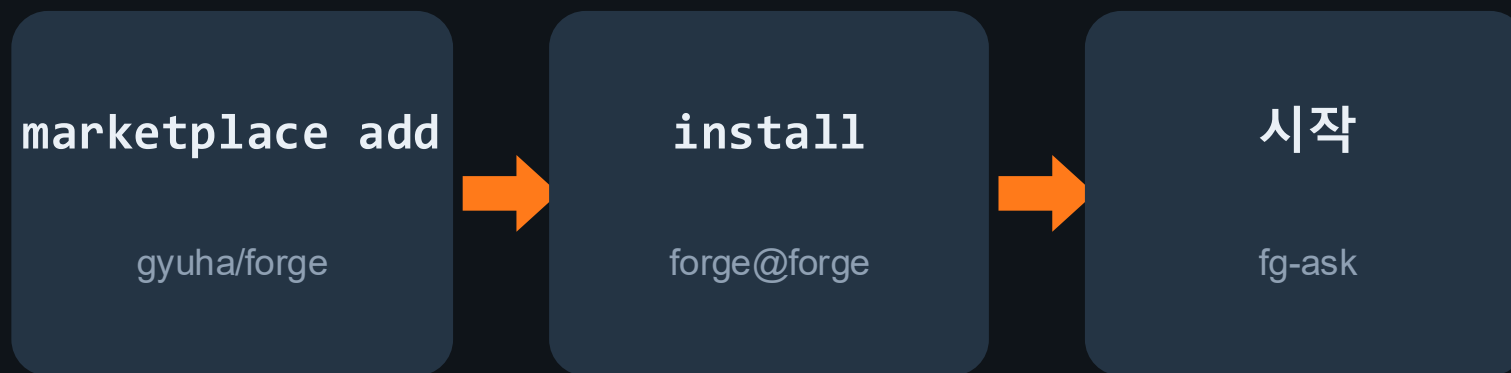
정답 소스

내용은 GitHub gyuha/forges의 README.ko.md와 docs/*.md가 기준입니다. 마지막에 출처가 있습니다.

배경 이론

Loop Engineering·Dynamic Workflow 이론 강의는 빼고, forge 명령을 이해하는 데 필요한 한 줄 정의만 둡니다.

| 설치와 업데이트

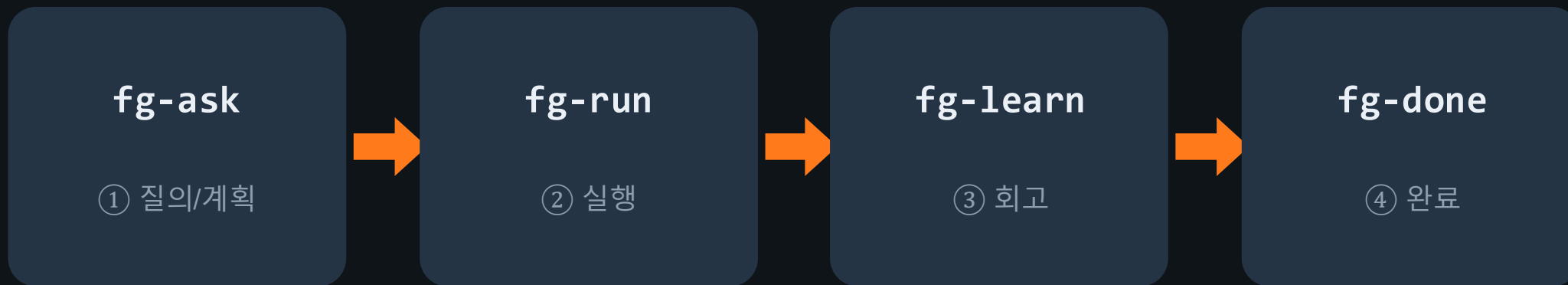


| 사실 3개만 쓰면 된다



다음 단계 자동 = 검증 → 회고 또는 봉인

한 바퀴의 의미



그릴링·대화형 → Dynamic Workflow → 문서 반영 → 봉인·재실행 방지

신뢰 사다리 — L1 / L2 / L3

L1 관찰

`/fg-status`

읽기 전용. 현황과 다음 단계를 보여주기만. 결정 전 판을 본다

L2 보조

`/fg-next`

다음 한 단계만 실행하고 멈춘다.
단계 사이에 당신이 운전석에 남는다

L3 무인

`/fg-loop` `/fg-next all`

루프를 통째로 완료까지 주행. 벽에서 멈춰 full-context로 인계

| .forge 상태 흐름

- backlog/ — 그릴링된 대기 계획들
- plan.md / run.md / STATUS.md — 활성 슬롯 (항상 1개)
- executed/ — 실행됐지만 회고 대기
- done/ — 봉인된 작업 이력



활성 슬롯: backlog → plan/run/STATUS → executed → done

왜 상태를 파일로 남기나

독립 호출

각 스킬을 따로 불러도 파일로 흐름이 이어진다. 세션 메모리에 의존하지 않는다.

재진입

다음 날 다시 커도 `.forge/`를 읽어 어디까지 했는지 안다. `fg-status`가 판을 보여준다.

재실행 방지

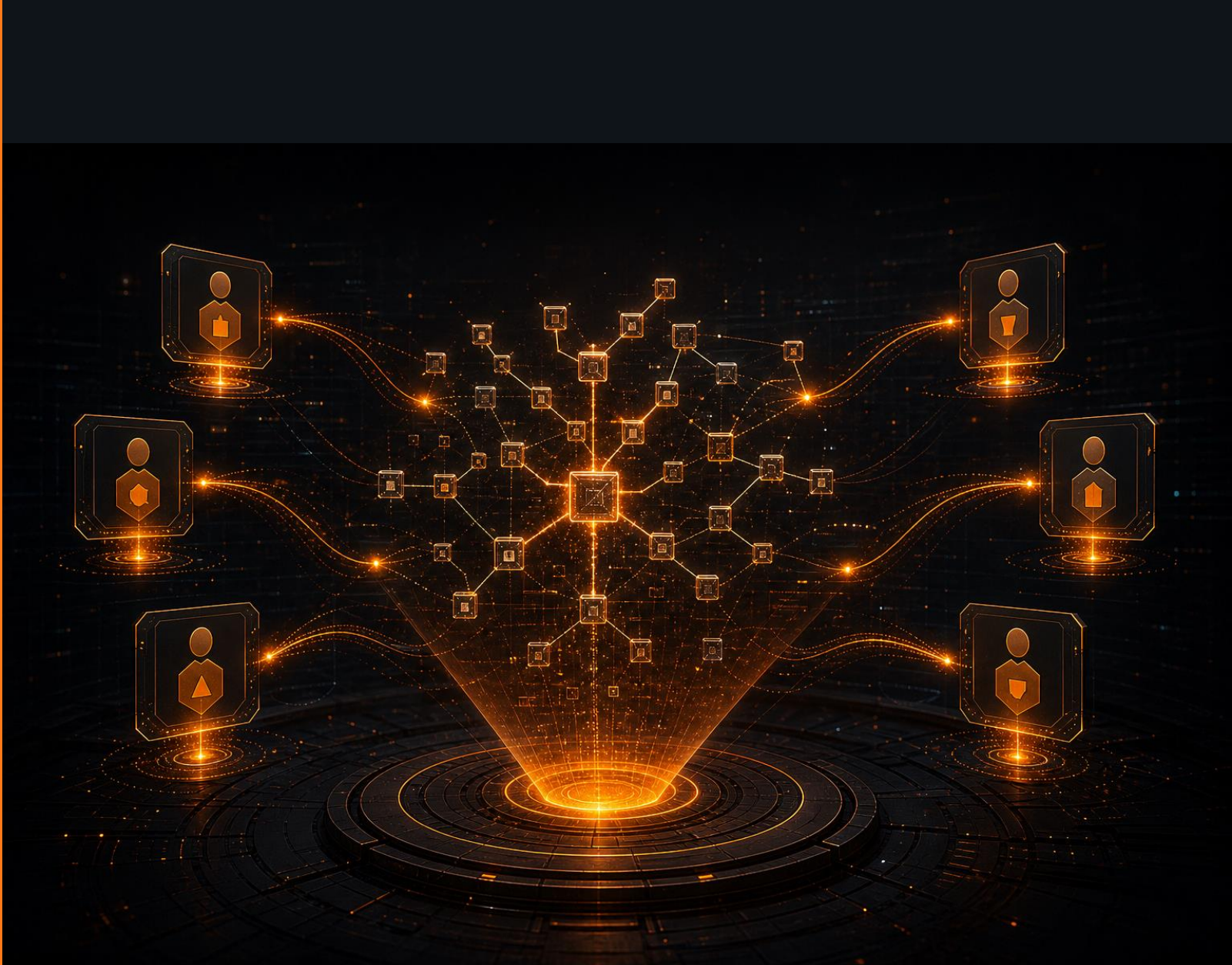
봉인(done)된 작업은 다시 실행되지 않는다. 같은 작업 두 번 실행을 구조적으로 막는다.

검증 게이트

봉인 전 `verified:` 가 기록돼야 한다 — 검증 없이 봉인할 수 없다 (`no-seal-without-verification`).

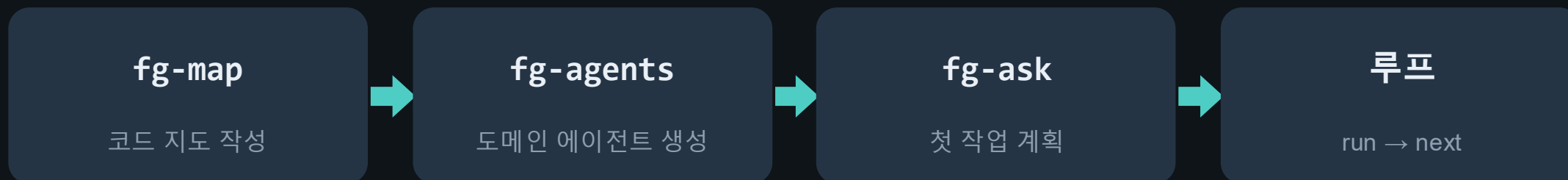
상황 1

처음 셋업



상황 1 — 처음 셋업

상황 새 프로젝트 / 코드베이스에 forge를 처음 붙인다



예시

예: 2만 줄짜리 레거시 레포. fg-map이 .forge/codebase/에 구조 지도를 만들고, fg-agents가 .claude/agents/에 맞춤 역할 카드를 만든다. 그 다음 fg-ask로 첫 작업을 잡는다.

⚠ 주의 — fg-agents 카드는 세션 시작 시에만 로드된다. 생성 뒤 한 번 재시작해야 fg-run이 호출한다 (ADR-0024).

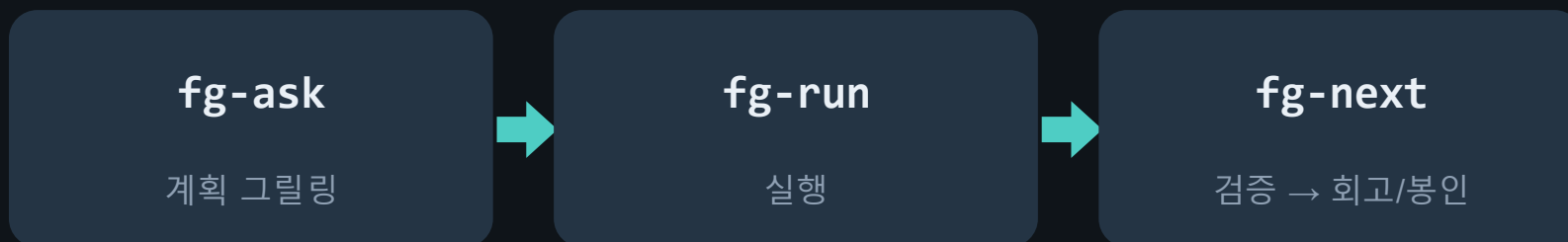
상황 2

일상 작업



상황 2 — 일상 작업

상황 버그 수정, 작은 기능 추가 같은 보통 규모의 작업



예시

예: '로그인 직후 간헐적 500 에러'. fg-ask가 한 질문씩 재현 조건·완료 기준·비목표를 못 박고 계획을 만든다. fg-run이 Dynamic Workflow로 수정·테스트하고, fg-next가 검증→회고로 이어간다.

⚠️ 팁 — fg-ask에서 용어·완료 기준·비목표가 정리된 계획이 run의 정답 소스가 된다. 계획이 엉망이면 실행도 엉망이다.

상황 3

사소한 1회 변경



상황 3 — 사소한 1회 변경

상황 오타, 문구 수정, 버전 범프 — 형식 산출물이 필요 없는 1회성

fg-quick

가볍게 그릴링



바로 실행

형식 산출물 없음

예시

예: README의 오타 한 글자, package.json 버전 1.2.3 → 1.2.4. fg-quick이 가볍게 확인만 하고 바로 고친다.
.forge/quick/LOG.md에 한 줄만 남는다.

⚠️ 탈출 조건 — fg-quick 도중 비-trivial로 드러나면 fg-ask로 bail한다. 사소한 줄 알았는데 버그가 깊으면 정식 루프로 옮긴다.

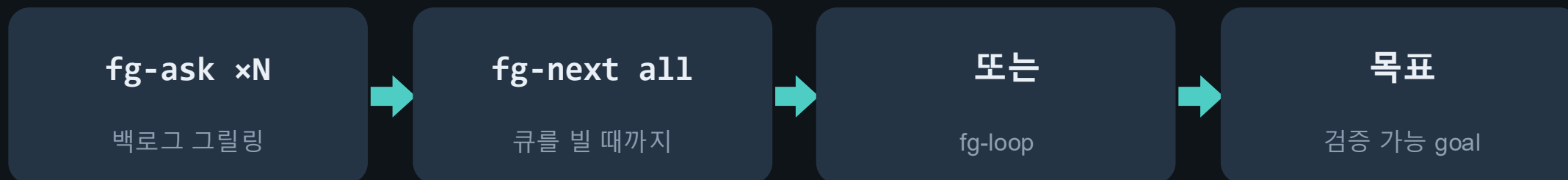
상황 4

무인 주행 완료까지



상황 4 — 무인 주행

상황 계획을 여러 개 쌓아두고 끝까지, 또는 기계 검증 가능한 목표를 수렴시킨다



예시

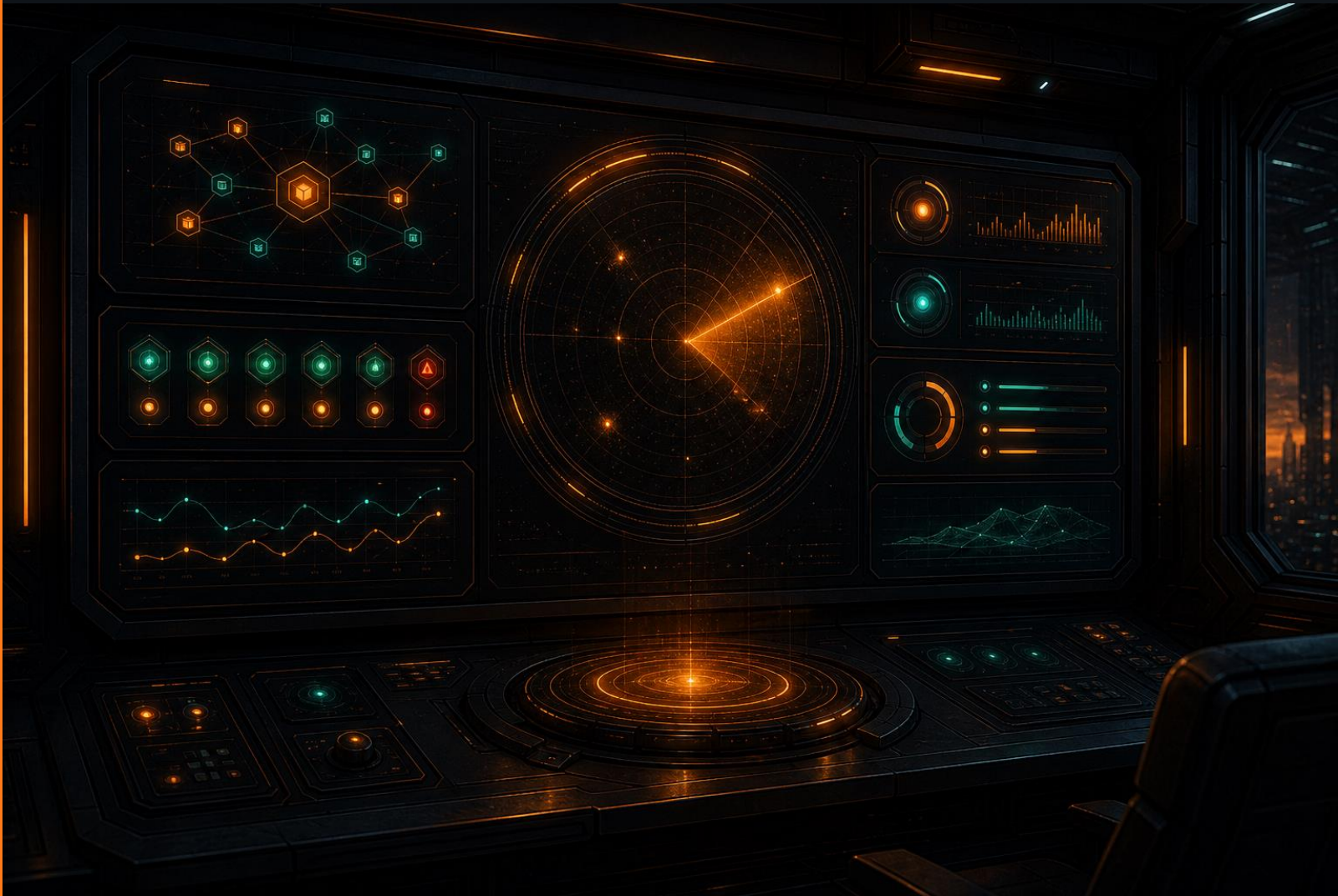
예 A — fg-next all: 주말에 작업 5개를 미리 그릴링해 두면 월요일에 한 명령으로 큐를 비운다.

예 B — fg-loop: '모든 테스트가 통과한다'를 정지 조건으로 못 박고, 실패 시 한정된 fix-forward를 스스로 만들며 수렴한다.

⚠️ **벽** — 검증 실패/불가, 진짜 fork, 무진전에서 멈춘다. fg-loop은 추가로 체크 간 tension 진동·비가역 액션에서도 멈춘다. 멈추면 full-context로 인계.

상황 5

재진입 / 점검



상황 5 — 재진입 / 점검

상황 어디까지 했는지 모를 때, 또는 상태가 건강한지 점검할 때



예시

예: 월요일 아침. fg-status를 치면 '작업 #3 활성화, 회고 대기'라고 알려준다. 상태가 이상하다 싶으면 fg-doctor가 고아 run.md·손상된 STATUS·slug 불일치 같은 위반을 severity별로 보고한다.

⚠ 구분 — fg-status는 '어디까지 했나'를, fg-doctor는 '상태가 건강한가'를 본다. 둘 다 읽기 전용이라 아무것도 쓰지 않는다.

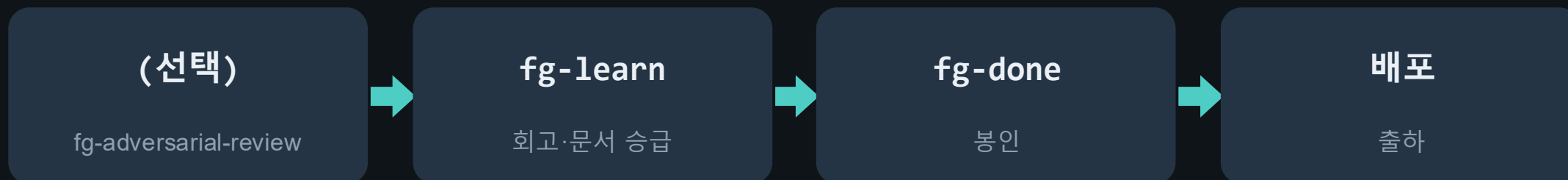
상황 6

마무리 / 배포



상황 6 — 마무리 / 배포

상황 실행이 끝난 작업을 회고하고 봉인하고 출하한다



예시

예: 결제 코드를 건드린 작업. 봉인 전 fg-adversarial-review가 '결과가 틀렸다고 가정하고' 6개 렌즈로 허점을 찾는다. findings 중 수정 필요 건은 fix-forward plan이 되고, 설계 결함은 fg-ask 재그릴링으로 돌아간다.

⚠ 선택 — 적대적 리뷰는 봉인 게이트가 아니다. 중요·위험 변경에만, 무인 주행에서는 항상 skip한다.

상황 7

유지보수



상황 7 — 유지보수 (루프 밖 유틸리티)

유틸리티	언제	하는 일
<code>fg-cleanup</code>	ADR이 쌓였을 때	오래된/대체된 ADR을 retired/로 은퇴 — 번호 불변·삭제 없음
<code>fg-merge</code>	브랜치 머지 후	<code>.forge/branch/<branch>/</code> 를 <code>.forge/</code> 로 통합 — ADR 재부여·용어 병합
<code>fg-drop</code>	미완 작업 버릴 때	backlog/활성/executed/멈춘 루프 폐기 — 하드 삭제 또는 보관
<code>fg-tdd</code>	테스트 우선 원할 때	config.json의 TDD 모드 토글 — ask가 기본 답으로 묻고 run이 test-first
<code>fg-eco</code>	토큰 절약 원할 때	eco 토글 — 위임 서브에이전트를 sonnet으로 캡 + 코드 단순성 규율
<code>fg-statusline</code>	상태바 원할 때	터미널에 forge 루프 진행 상태를 띄우는 스크립트 설치

어떤 차선을 고를까

상황	차선	명령
사소한 1회성 (오타·버전)	경량	/fg-quick
보통 작업, 한 단계씩 지켜볼 때	정식 루프	/fg-ask → /fg-run → /fg-next
그릴링한 백로그를 끝까지	무인 (큐)	/fg-next all
검증 가능 goal로 수렴	무인 (goal)	/fg-loop
길 잃었을 때 (판만 본다)	관찰	/fg-status

fg-ask에서 좋은 계획을 만드는 법

용어

애매한 단어를 정확한 용어로 못 박는다 — '계정'이 Customer인지 User인지.

완료 기준

각 작업 조각에 관측 가능한 완료 조건을 단다 — '돌아간다'가 아니라 '테스트가 통과한다'.

비목표

이번에 안 할 것을 명시한다 — 범위 폭발을 막는 가장 강력한 도구.

조각(slice)

큰 작업을 독립적으로 검증·봉인 가능한 단위로 쪼갬다 — run의 작업 단위.

| fg-run 뒤에 확인할 것

plan vs actual

run.md에 계획과 실제의 차이(divergence)가 기록된다 — 회고의 원료.

검증 게이트

봉인 전 verified: 가 yes/skipped/n/a 중 하나여야 한다 — pending·failed는 봉인 불가.

STATUS.md

실행 직후 status: executed, verified: pending 으로 마킹된다 — 회고·봉인이 이어받는다.

divergence

낮으면 회고 skip + 봉인, 높으면 fg-ask 재그릴링 — 차이가 크면 배울 게 있는 것이다.

실패했을 때 — **failed**는 봉인 불가

failed

검증이 망가졌다

UAT가 돌았고 결과가 목표를 달성하지 못한다. pending(검증 안 함)과 다르다

경로 1

`fix-and-re-run`

고치고 의도적으로 재실행 — 새 `run.md`, 재검증. `failed` → `yes/n/a`로 바뀌어야 봉인

경로 2

재그릴링

`plan` 자체가 틀렸으면 `fg-ask`로 돌아가 계획을 다시 다듬는다

이미지와 도형을 섞어 설명한다

- 분위기/섹션 이미지는 codex-image로 — 텍스트 없이
- 명령·상태명·화살표는 PPTX 텍스트·도형 레이어로
- 이미지 안에 명령어를 박으면 깨지고 틀린다
- 흐름도는 '상황 → 명령 → 산출물 → 다음' 구조로



치트시트 1 — 상황 → 명령 → 다음

상황	명령	다음
처음 셋업	<code>/fg-map → /fg-agents → /fg-ask</code>	루프
일상 작업	<code>/fg-ask → /fg-run → /fg-next</code>	회고/봉인
사소한 1회	<code>/fg-quick</code>	끝
무인 (큐)	<code>/fg-ask xN → /fg-next all</code>	벽까지
무인 (goal)	<code>/fg-loop</code>	수렴/벽
재진입/점검	<code>/fg-status · /fg-next · /fg-doctor</code>	다음 단계
마무리/배포	<code>/fg-learn → /fg-done → 배포</code>	출하



치트시트 2 — 18개 스킬 전체 요약

스킬	단계	한 줄
fg-ask	① 계획	그릴링으로 계획 — 모든 작업의 시작
fg-run	② 실행	Dynamic Workflow로 실행
fg-learn	③ 회고	학습을 문서로 승급
fg-done	④ 완료	한 바퀴 정리·봉인
fg-map	유틸	코드베이스를 지도로
fg-quick	유틸	사소한 작업 경량 차선
fg-status	유틸	현황 읽기 전용 표시
fg-next	유틸	다음 단계 도출·실행
fg-loop	유틸	goal 주도 무인 루프
fg-tdd / fg-eco	토글	TDD 모드 / 토큰 절약 모드
fg-merge / fg-cleanup	정비	브랜치 통합 / ADR 은퇴
fg-drop / fg-doctor	정비	미완 폐기 / 무결성 점검
fg-adversarial-review	리뷰	봉인 전 선택적 적대 검토
fg-agents / fg-statusline	생성	도메인 에이전트 / 상태바

흔한 실수 3개

1 계획 없이 run

run은 plan을 정답 소스로 삼는다.
계획 없이 run하면 엉뚱한 게 나온다 — 반드시 fg-ask부터.

2 상태 무시

.forge/를 안 보면 재진입·재실행
위험이 생긴다. fg-status로 먼저
판을 본다.

3 failed 봉인 시도

failed는 봉인 불가. skipped로 속
이지 말고 fix-and-re-run 또는 재
그릴링으로 간다.

출처

README.ko.md

빠른 시작 · 차선 고르기 · 사용 시나리오 · 카탈로그 · 설치 — github.com/gyuha/forge

docs/skills.md

18개 스킬 상세 — 입력·출력·다음·트리거·근거 ADR

docs/state-contract.md

.forge/ 디렉터리 · 상태 계약 · 검증 게이트 · 회고 skip 규칙

docs/forge-vs-loop-engineering.md

L1/L2/L3 신뢰 사다리 · 두 기동 · fg-loop 경화 근거